Contextual Multi-Armed Bandits for Ad Format Optimisation

1. Project Objective

In digital advertising marketplaces, Click-Through Rate (CTR) is a fundamental metric used to evaluate user engagement and advertising effectiveness. It directly influences auction outcomes, advertiser returns, and platform revenue, making it a critical lever in marketplace design. Given the increasing complexity of digital platforms and the availability of rich user and contextual data, understanding the determinants of CTR is essential for optimising ad placement strategies.

The objective of this project is to identify the key features that drive CTR and to evaluate how these insights can inform optimal ad placement decisions. In doing so, the project aims to bridge predictive analytics with actionable managerial strategies, enabling platforms to enhance both user experience and monetisation.

2. Dataset Overview

This study utilises the Avazu click-through rate (CTR) prediction dataset, available on Kaggle at: https://www.kaggle.com/competitions/avazu-ctr-prediction/overview.

With approximately 40 million records in the original CSV file, the dataset is quite large. To handle this volume, we leveraged the Polars library, using its lazy evaluation features to perform memory-efficient data cleaning and exploration without the need to load the entire dataset at once.

Following a filter for site_id=85f751fd, the refined data consists of 14,596,137 records. The individual features are detailed in the "feature summary table" located in the appendix. This click data spans 10 days in October 2014, specifically from the 21st through the 30th.

3. Data Cleaning & First Feature Selection

The dataset contains 6 distinct banner positions. Positions 2, 3, 5, and 7 account for only 0.35% of logged impressions combined (23, 802, 5,778, and 43,577 records, respectively) and were excluded. Under the offline replay framework (Li et al., 2012), meaningful policy updates are statistically infeasible with so few impressions. Furthermore, oracle predictions

for rare positions are unreliable due to insufficient training examples. The study focuses on positions 0 and 1, accounting for 99.65% of impressions.

Feature selection was performed using a filter-based approach, evaluating each feature's mutual information (MI) with the target variable click, which is a model-agnostic method. MI quantifies the reduction in uncertainty about the target given knowledge of a feature, making it a model-agnostic measure of predictive utility (Guyon & Elisseeff, 2003).

The following MI scores were obtained from a 100,000-row sample, see appendix: C15 and C16 were excluded on two complementary grounds: near-constant value distributions (96.51% and 97.81% of records sharing a single value) and MI scores that were an order of magnitude lower than those of all other retained features. All remaining features were retained.

Notably, banner_pos has the lowest MI among all features (0.000285), reflecting the heavily imbalanced exposure of positions in the logged data; position 0 accounts for 91.5% of impressions. A feature with near-zero MI with the target cannot be optimised through passive observation alone. This directly motivates the use of an MAB framework: active exploration is necessary to learn the true reward of underrepresented positions that logged data cannot adequately represent.

4. Methodology

4.1 Offline Policy Evaluation & the Counterfactual Problem

Evaluating a MAB policy requires estimating regret — the difference between the reward of the chosen arm and the reward of the optimal arm. In logged observational data, only the outcome of the displayed banner position is observed; outcomes for unchosen positions are unobserved counterfactuals.

To evaluate CTR, we adopt the replay evaluation method introduced by Li et al. (2012). At each step, the MAB selects a banner position; if it matches the position logged in the data, the interaction is recorded, and the reward (click or no click) is observed. Unmatched interactions are discarded. This ensures that only unbiased, on-policy observations are used to estimate CTR.

To evaluate regret, an oracle is required — a prediction model that estimates the expected click probability for every banner position given the observed context, approximating the counterfactual outcomes unavailable in the log. Regret for each matched interaction is then computed as the difference between the oracle's predicted CTR for the optimal position and that of the MAB's chosen position.

4.2 Oracle: XGBoost CTR Predictor

Tree-based models were preferred over linear models due to expected non-linear interactions among anonymous categorical features, and over neural networks on grounds of computational efficiency relative to marginal accuracy gain. Among tree-based methods, XGBoost and Random Forest were selected for comparison, as tree ensemble methods have been empirically shown to dominate tabular classification benchmarks (Fernández-Delgado et al., 2014). XGBoost (depth 6) achieved a validation log-loss of 0.3653, outperforming the best Random Forest configuration (n_estimators=100, log-loss=0.3671). XGBoost was therefore selected as the oracle for all downstream regret computations, consistent with its demonstrated state-of-the-art performance on large-scale tabular datasets (Chen & Guestrin, 2016).

4.2.1 Model Training

The oracle was trained across max_depth $\in$ {6, 8, 10}, evaluated on the held-out validation set (day 30). Depth 6 achieved the lowest validation log-loss (0.3653); deeper models showed degraded validation performance, indicating overfitting at higher capacity. The temporal split: days 21–29 for training, day 30 for validation. This reflects realistic deployment conditions where the model must generalise to future unseen data.

4.2.2 Handling Class Imbalance in Oracle Training

Training CTR prediction tasks are often highly imbalanced because positive click events are rare relative to non-click events. The XGBoost documentation (n.d.) explicitly discusses this issue and notes that the appropriate treatment depends on the modelling objective: if the goal is ranking performance, adjusting class weights via scale_pos_weight may improve AUC; however, if the goal is to produce well-calibrated probability estimates, such rebalancing can distort output probabilities.

Since our oracle is used to estimate click probabilities for downstream regret computation, we do not introduce class rebalancing to preserve probability calibration. In our dataset, the negative-to-positive ratio is approximately 7.5:1, which we consider moderate and unlikely to bias the model's probability estimates severely.

4.3 Bandit Algorithm Implementations

Three policies are evaluated: two context-free baselines (Epsilon-Greedy and UCB) and one contextual algorithm (LinUCB). All three operate under the offline replay framework described in Section 4.1. The algorithms are selected to mirror the progressive complexity introduced in the MAB framework covered in the lecture: from the simplest exploration strategy (Epsilon-Greedy) to a frequentist confidence-bound approach (UCB), to a personalised contextual extension (LinUCB).

4.3.1 Baseline 1: Epsilon-Greedy

Motivation and Design.

Epsilon-Greedy is the simplest algorithm for balancing exploration and exploitation in the MAB setting. At each step the agent must decide between *exploring* - randomly selecting any arm to gather information about under-played options and *exploiting* - selecting the arm with the highest estimated mean reward. Epsilon-Greedy resolves this dilemma with a fixed probability parameter ε:

With probability ε: randomly select any arm (exploration)

With probability $1 - \varepsilon$: select the arm with the highest estimated mean CTR (exploitation)

Update Rule.

After each matched interaction, the estimated reward for the chosen arm is updated incrementally. The running mean is maintained as:

$$\hat{\mu}_a \leftarrow (\hat{\mu}_a \cdot N_a + r) / (N_a + 1), \quad N_a \leftarrow N_a + 1$$

where r is the observed reward, and N_a is the number of times arm a has been selected. This incremental update is equivalent to a running mean and avoids storing the full reward history.

Hyperparameter.

ε is fixed at 0.05, meaning the agent explores randomly 5% of the time. This is a standard choice for stationary reward environments; higher values introduce unnecessary noise once reward estimates have stabilised. The exploration rate ε can also be decayed over time (εt) for longer horizons, but a fixed rate is appropriate here given the 10-day observation window.

Limitation.

A key limitation is that exploration is entirely random - the agent has no preference for under-explored arms and ignores all visitor context entirely when making decisions. The algorithm finds a single globally optimal arm rather than a visitor-specific one.

4.3.2 Baseline 2: UCB

Motivation and Design.

UCB implements the principle of *optimism in the face of uncertainty*. Rather than exploring randomly, UCB constructs a confidence interval around each arm's estimated mean and selects the arm with the highest upper confidence bound. This means an arm with a lower estimated mean may still be selected if its confidence interval is wide enough - reflecting genuine uncertainty rather than poor performance.

Formal Selection Rule.

Following the UCB algorithm at each step, the arm is selected by:

$$a_t = \text{argmax} \left[S_a/N_a + \sqrt{(2 \log t / N_a)} \right]$$

where S_a/N_a is the estimated mean reward (exploitation term) and $\sqrt{(2 \log t / N_a)}$ is the confidence bonus (exploration term). The bonus is large when an arm has been tried infrequently and shrinks as more observations accumulate. The coefficient 2 in the bonus term reflects the 'correct' uncertainty in the frequentist setting - it is theoretically derived, not a tuneable hyperparameter.

Initialisation.

Following the formal algorithm, each arm is played exactly once during initialisation before the UCB selection rule takes over. This ensures that the confidence bound is defined for all arms from the start.

Limitation.

Like Epsilon-Greedy, UCB maintains no representation of visitor context. It identifies a single globally optimal banner position across all visitors, rather than adapting to individual session characteristics.

4.3.3 LinUCB - Contextual MAB

Motivation and Design.

LinUCB is a natural extension of UCB to the contextual MAB setting. In the contextual framework, at each time t, the agent observes a context vector x_t representing relevant information about the current visitor and environment. Rewards depend on both the chosen arm and the observed context. LinUCB exploits this structure by assuming that the expected reward is a *linear function* of the context vector:

$$E[r \mid a, x] = x^T \hat{\theta}_a$$

where x is the context vector, and $\hat{\theta}_a$ are the learned weight parameters for arm a. The arm selection rule extends UCB by adding a context-aware confidence bonus:

$$a_t = \text{argmax} \left[x^T \hat{\theta}_a + \alpha \sqrt{(x^T A_a^{-1} x)} \right]$$

The first term is the predicted CTR for this specific visitor; the second term is the uncertainty bonus scaled by the exploration parameter α. The matrix A_a accumulates the outer products of observed context vectors for arm a and is updated via the Sherman-Morrison formula for computational efficiency, avoiding a full matrix inversion at each step.

Context Feature Construction.

The lecture identifies context vector construction as a critical practical step, noting that features should capture information relevant to decision-making and must be available in real-time. The feature vector x is constructed from the following visitor-level signals:

Feature	Type	What It Captures
device_type	Categorical (OHE)	Mobile, tablet, desktop, connected TV - device context affects ad format suitability
device_conn_type	Categorical (OHE)	WiFi, 3G, 4G - connection type affects load behaviour and user intent
hour_of_day	Numeric (standardised)	Time of day derived from the hour field - captures temporal patterns in click behaviour
app_category	Categorical (OHE)	Content category of the app - captures advertised content groupings
C1	Categorical (OHE)	Low-cardinality anonymous categorical feature with meaningful MI signal
C20	Categorical (OHE)	Anonymous categorical feature retained based on MI score
C21	Categorical (OHE)	Anonymous categorical feature retained based on MI score

Categorical features are one-hot encoded, and the numeric feature (hour_of_day) is standardised, producing a **253-dimensional** context vector. High-cardinality identifiers (device_id, device_ip) are excluded on dimensionality reduction - these would inflate the feature space without adding a reliable signal. The selected features represent meaningful visitor-level context, consistent with recommendation system examples (e.g., user demographics, device type, location).

Why LinUCB over Contextual Thompson Sampling?

Both LinUCB and Contextual Thompson Sampling are extensions to the contextual MAB setting. LinUCB is selected here for two reasons: first, UCB is a deterministic algorithm, which makes offline replay evaluation more straightforward; second, TS requires sampling from posterior distributions and Bayesian updating, which is computationally more expensive at 500,000 evaluation records. In more complex environments, UCB often requires more

fine-tuning to achieve similar performance - hyperparameter search over α directly addresses this.

4.3.4 Alpha Hyperparameter Tuning for LinUCB

Role of Alpha.

The exploration parameter α scales the confidence bonus in the LinUCB selection rule. A higher α widens the confidence interval, increasing exploration; a lower α narrows it, favouring exploitation of current estimates. The size of the confidence interval reflects how uncertain the agent is about each arm - the bonus is larger when uncertainty is higher. Unlike UCB's theoretically derived coefficient of $\sqrt{2}$, α in LinUCB must be tuned empirically because the scale of the confidence term depends on the geometry of the 253-dimensional feature space.

Search Procedure.

Four candidate values were evaluated - $\alpha \in \{0.1, 0.5, 1.0, 1.5\}$ - using the offline replay evaluator on the 500,000-record evaluation sample. Final CTR and total oracle-based regret are reported for each:

Alpha (α)	Final CTR	Total Regret	Interpretation
0.1	0.21639	769.13	Strong exploitation - insufficient exploration of arm-context combinations
0.5	0.22057	533.76	Moderate exploration - improvement over low alpha
1.0	0.22344	497.15	Higher exploration - continued improvement in both metrics
1.5	0.22482	493.63	Highest exploration - best CTR and lowest regret

Selection of α = 1.5.

Two consistent monotonic trends emerge from the search results. First, CTR increases with α across all candidate values - greater exploration leads to better arm selection as the algorithm gathers richer context-reward observations across the feature space. Second, total regret decreases with α, indicating that the additional exploration cost of higher α is more than recovered through improved subsequent decisions.

$\alpha = 1.5$ is selected as the optimal value, achieving the highest final CTR (0.22482) and lowest total regret (493.63). It is notable that this value approaches UCB's theoretical coefficient of $\sqrt{2} \approx 1.41$. The bonus term should reflect 'correct uncertainty' in the estimate - in a high-dimensional feature space with substantial context heterogeneity across 253 dimensions, a relatively wide confidence interval is necessary for the linear model to adequately explore arm-context combinations it has not yet encountered.

4.4 Summary Comparison of Algorithms

Algorithm	Type	Exploration Mechanism	Uses Context?	Key Parameter
Epsilon-Greedy	Standard MAB	Random with probability ε	No	$\varepsilon = 0.05$ (fixed)
UCB	Standard MAB	Confidence bound $\sqrt{(2 \log t / N_a)}$	No	None (theoretically derived)
LinUCB	Contextual MAB	Context-aware bound $\alpha\sqrt{(x^T A_a^{-1} x)}$	Yes (253-dim vector)	$\alpha = 1.5$ (tuned)

5. Results

5.1 Oracle Model Performance

The oracle comparison shows that XGBoost achieves a validation log-loss of **0.3653**, outperforming Random Forest at **0.3671**. Although the absolute difference appears small, it is consistent and meaningful given the scale of the dataset and the sensitivity of probability estimation tasks. This confirms that XGBoost provides better-calibrated click probability estimates and is therefore suitable as the oracle model for downstream regret computation.

5.2 Feature Efficiency for Contextual Learning

The likelihood ratio test (LRT) signal per dimension reveals strong heterogeneity in feature importance. The most informative feature is **device connection type**, with a signal-per-dimension value of approximately **519.5**, far exceeding all other features. This is followed by **C18 (223.6)** and **app category (173.8)**. In contrast, features such as **hour of day (26.0)** and **C20 (21.3)** contribute relatively little signal per dimension.

Importantly, high-cardinality features such as C21 (d=50) and C19 (d=58) exhibit moderate signal but lower efficiency per dimension, suggesting diminishing returns as the feature space expands. These results justify the use of a contextual bandit, as meaningful variation exists across user contexts, particularly in connection type and app-level behaviour.

5.3 LinUCB Hyperparameter (α) Tuning

The performance of LinUCB is highly sensitive to the exploration parameter α. The results show that **α = 0.5** delivers the best balance between exploration and exploitation.

In terms of CTR, α = 0.5 achieves the highest final value of approximately **0.2216**, significantly outperforming α = 0.1 (0.1738), α = 1.0 (0.1712), and α = 1.5 (0.1718). This represents a substantial improvement of roughly **+4.8 percentage points** over the weaker configurations.

From a regret perspective, α = 0.5 also achieves the lowest average regret trajectory over time. Higher values of α (e.g., 1.5) exhibit unstable early behaviour with high initial regret due to excessive exploration, while lower values (α = 0.1) under-explore and converge prematurely to suboptimal policies.

5.4 Final Algorithm Comparison

Using the optimal configuration (LinUCB with α = 0.5), the final comparison across algorithms shows a clear performance gap.

LinUCB achieves a final CTR of **0.2216**, compared to **0.1302** for Epsilon-Greedy and **0.1285** for UCB. This represents an improvement of approximately **70% relative uplift in CTR**, highlighting the value of contextual learning.

In terms of regret, LinUCB again outperforms both baselines. The final average regret is **0.00456** for LinUCB, compared to **0.00479** for UCB and **0.00481** for Epsilon-Greedy. While the absolute differences appear small, they accumulate significantly over millions of interactions, as reflected in the cumulative regret curves.

The regret trajectories further show that LinUCB stabilizes more quickly and maintains consistently lower regret over time, whereas both Epsilon-Greedy and UCB exhibit slower convergence and higher long-run regret accumulation.

6. Insights

6.1 Context Drives Performance Gains

The most striking result is the magnitude of improvement achieved by LinUCB over non-contextual methods. The large CTR gap confirms that user response to ad placement is

highly dependent on contextual factors, particularly connection type and app category. Ignoring these features leads to systematically suboptimal decisions.

6.2 Optimal Exploration is Critical

The alpha tuning results demonstrate that performance is highly sensitive to the exploration parameter. Both under-exploration ($\alpha = 0.1$) and over-exploration ($\alpha \geq 1.0$) degrade performance. The optimal value ($\alpha = 0.5$) achieves a balance where the algorithm explores sufficiently early on but quickly shifts toward exploitation.

6.3 Feature Efficiency Matters More Than Feature Count

The LRT analysis shows that not all features contribute equally. A small number of highly informative features dominate performance, while many high-cardinality features add limited incremental value. This suggests that careful feature selection is more important than simply increasing model complexity.

6.4 Small Regret Differences Scale to Large Impact

Although the differences in average regret appear numerically small, they translate into large cumulative differences over hundreds of thousands of impressions. This reinforces why regret minimisation is a critical metric in high-frequency decision environments like digital advertising.

7. Managerial Implications

7.1 Moving Beyond A/B Testing

The results highlight the limitations of static A/B testing in dynamic environments. Unlike A/B tests, which allocate traffic inefficiently during experimentation, bandit algorithms continuously optimise decisions, reducing opportunity cost and improving outcomes in real time.

7.2 Personalisation as a Core Capability

The strong performance of LinUCB demonstrates that personalisation is not optional but essential. Platforms that fail to incorporate contextual information risk significant performance losses, both in user engagement and revenue generation.

7.3 Efficient Allocation of Advertising Inventory

Lower regret directly translates into more efficient allocation of impressions to high-performing placements. This improves advertiser ROI and increases platform monetisation, aligning incentives across stakeholders.

7.4 Practical Deployment Strategy

The results support a scalable deployment model where a separate contextual bandit is trained per site. This allows each site to learn its own user behaviour patterns while maintaining a consistent optimisation framework across the platform.

8. Business Value and Impact

8.1 Revenue and User Engagement

Boosting CTR from 0.13 to 0.22 significantly increases revenue per impression. Additionally, relevant placements enhance user experience, fostering long-term platform loyalty.

8.2 Operational Efficiency

LinUCB minimizes experimentation costs by converging rapidly to optimal policies, reducing wasted impressions on suboptimal ads.

8.3 Competitive Advantage

Real-time adaptation through contextual bandits provides a scalable differentiator in algorithmic marketplace design.

9. Conclusion

This study demonstrates that contextual multi-armed bandit algorithms, particularly LinUCB, provide significant improvements in ad placement optimisation compared to traditional non-contextual approaches.

The results show that incorporating contextual information leads to substantial gains in CTR and reductions in regret, while careful tuning of exploration parameters is essential for optimal performance. The findings also highlight the importance of feature efficiency, with a small number of high-signal features driving most of the performance gains.

From both a technical and business perspective, the adoption of contextual bandit algorithms represents a meaningful advancement over static experimentation methods. By enabling continuous learning and real-time adaptation, these methods allow platforms to better match user preferences, improve engagement, and maximise revenue.

Overall, the project reinforces the central role of adaptive, data-driven algorithms in modern online marketplaces and demonstrates their practical value in large-scale digital advertising systems.

References

1. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
2. Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. Journal of Machine Learning Research, 3, 1157–1182.
3. He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., & Quiñonero Candela, J. (2014). Practical lessons from predicting clicks on ads at Facebook. Proceedings of the 8th International Workshop on Data Mining for Online Advertising (ADKDD'14), 1–9.
4. Li, L., Chu, W., Langford, J., & Schapire, R. E. (2012). A contextual-bandit approach to personalized news article recommendation. Proceedings of the 19th International Conference on World Wide Web (WWW 2010), 661–700.
5. XGBoost Development Team. (n.d.). Notes on parameter tuning. XGBoost Documentation. https://xgboost.readthedocs.io/en/stable/tutorials/param_tuning.html

Appendix

Feature summary table

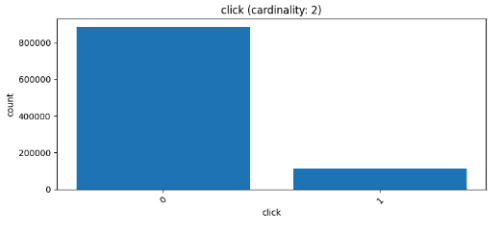
Name	Type	Cardinality	Keep?	Reason
id	u64	14596137	Drop	Row identifier has no predictive signal
click	i64	2	Keep	Target variable and MAB reward signal
hour	i64	240	Keep	Used to derive time of day and temporal train/val split
C1	i64	7	Keep	Low cardinality anonymous categorical feature
banner_pos	i64	6	Keep	The arm banner position is being optimized, filtered to positions [0,1] only due to insufficient impressions for reliable evaluation of rare positions.
site_id	str	1 (after filter)	Drop	Used as a filter to scope the study to one site, not a feature
site_domain	str	1 (after filter)	Drop	Zero variance after filtering to one site
site_category	str	1 (after filter)	Drop	Zero variance after filtering to one site
app_id	str	8551	Drop	Anonymized hash with 8551 unique values, a higher cardinality version of the same signal captured by app_category
app_domain	str	559	Drop	Anonymized hash with 559 unique values, a higher cardinality version of the same signal captured by app_category
app_category	str	36	Keep	Captures advertised content groupings at a computationally efficient granularity.
device_id	str	1737392	Drop	User fingerprint anonymized, computationally heavy, could be noisy
device_ip	str	2724972	Drop	User fingerprint anonymized, computationally heavy, could be noisy
device_model	str	6918	Drop	Anonymized hash has no semantic meaning, can be noisy; device_type already covers a meaningful context, which can be a better signal.
device_type	i64	4	Keep	Low cardinality meaningful device context
device_conn_type	i64	4	Keep	Low cardinality meaningful connection context

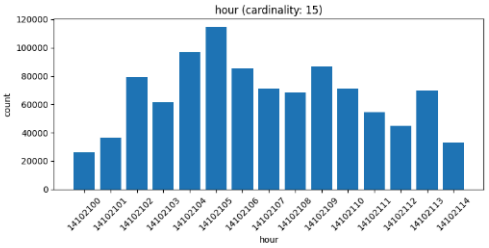
Name	Type	Cardinality	Keep?	Reason
C14	i64	2252	Keep	Anonymous categorical sufficient variance for context
C15	i64	8	Drop	Low cardinality anonymous categorical feature, but the MI value is very low
C16	i64	9	Drop	Low cardinality anonymous categorical feature, but it has a very centralised distribution with a very low MI value.
C17	i64	369	Keep	Anonymous categorical sufficient variance for context
C18	i64	4	Keep	Low cardinality anonymous categorical feature
C19	i64	63	Keep	Low cardinality anonymous categorical feature
C20	i64	170	Keep	Anonymous categorical sufficient variance for context
C21	i64	55	Keep	Low cardinality anonymous categorical feature

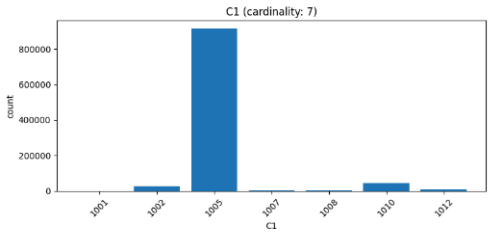
Feature and MI score

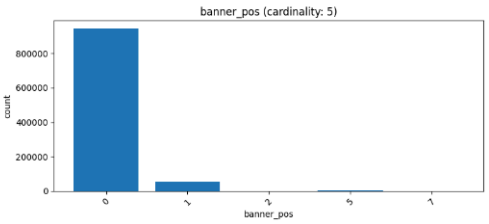
Feature	MI
C14	0.022633
C17	0.020974
C21	0.017645
C19	0.016165
C18	0.006919
C20	0.006082
app_category	0.004251
C1	0.003287
device_type	0.002760
hour	0.002180
device_conn_type	0.002175
C15	0.000584
C16	0.000509

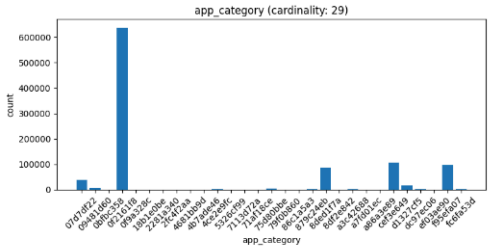
Feature	MI
banner_pos	0.000309

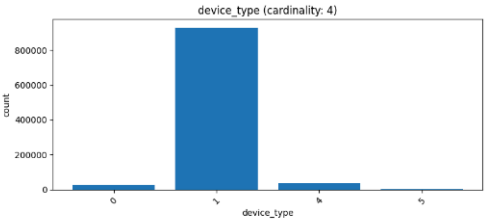

click (cardinality: 2)
count
click

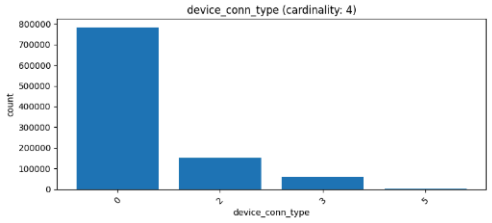

hour (cardinality: 15)
count
hour

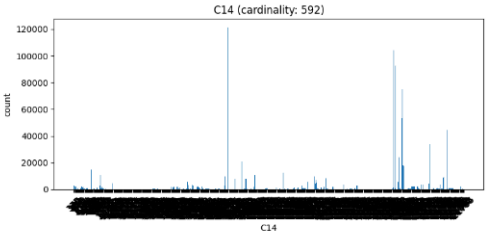

C1 (cardinality: 7)
count
C1

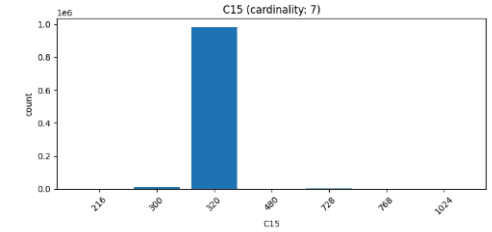

banner_pos (cardinality: 5)
count
banner_pos

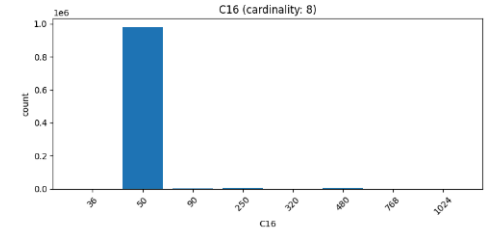

app_category (cardinality: 29)
count
app_category

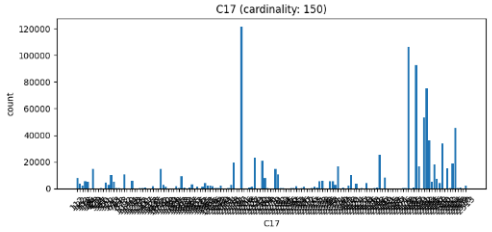

device_type (cardinality: 4)
count
device_type

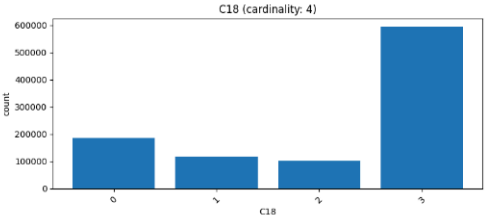

device_conn_type (cardinality: 4)
count
device_conn_type

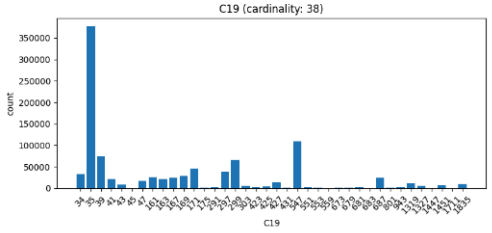

C14 (cardinality: 592)
count
C14

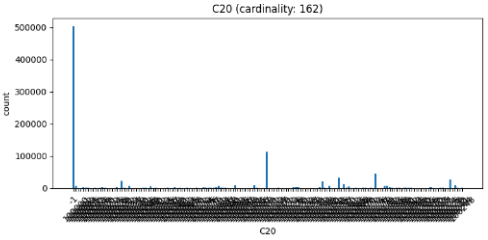

C15 (cardinality: 7)
1e6
count
C15

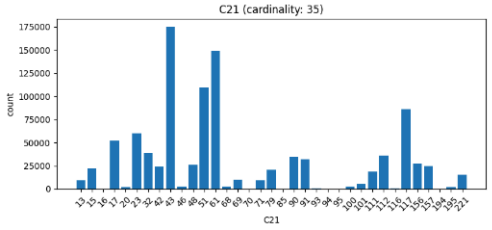

C16 (cardinality: 8)
1e6
count
C16


C17 (cardinality: 150)
count
C17


C18 (cardinality: 4)
count
C18


C19 (cardinality: 38)
count
C19


C20 (cardinality: 162)
count
C20


C21 (cardinality: 35)
count
C21